

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

GRUPO 2CM3

GESTOR DE CITAS MÉDICAS

Entrega Parcial 1 · Etapa Funcional

Materia: Paradigmas de Programación

Profesora: Martínez Hernández Guadalupe Ana Gabriela

Integrantes

Rosas Rios Luis Gabriel

Mendez Gamboa Santiago

Contreras Cruz Joseph

1. Recuperación del trabajo previo

El proyecto consiste en desarrollar un gestor de citas médicas que permita organizar la información de pacientes, doctores, horarios y citas. El sistema contempla operaciones de registro, consulta, agendado, cancelación y reprogramación, además de confirmaciones o notificaciones y protección de datos personales.

La propuesta tecnológica mantiene Python como lenguaje principal por su carácter multiparadigma. JavaScript se utiliza para la interacción del frontend, SQL queda destinado a la persistencia y HTML/CSS a la estructura y presentación de la aplicación web.

1.1 Requerimientos

ID	Requerimiento	Actor
R01	Registrar pacientes en el sistema	Recepción
R02	Registrar doctores y sus especialidades	Administrador
R03	Permitir iniciar sesión a los usuarios	Usuarios autorizados
R04	Mostrar los horarios disponibles de los doctores	Recepción / Paciente
R05	Permitir agendar una cita médica	Recepción
R06	Impedir agendar una cita cuando el doctor ya tenga otra cita en ese horario	Recepción
R07	Permitir cancelar una cita	Recepción / Administrador
R08	Permitir reprogramar una cita	Recepción / Administrador
R09	Mostrar al paciente sus citas próximas	Paciente
R10	Permitir al doctor consultar su agenda	Doctor
R11	Permitir al administrador agregar, modificar o eliminar usuarios	Administrador
R12	Enviar una confirmación cuando se registre una cita	Automático
R13	Mantener protegidos los datos personales de pacientes y doctores	Sistema
R14	Permitir al doctor notificar que no asistirá para cancelar su lista	Doctor

1.2 Roles y actores

Actor	Responsabilidades principales
Administrador	Gestiona usuarios y operaciones administrativas.
Doctor	Consulta agenda y horarios; puede notificar que no asistirá.
Paciente	Consulta sus citas próximas y la información necesaria para su atención.
Recepción	Registra pacientes y realiza el agendado y gestión de citas.
Personal autorizado	Accede a funciones asignadas por el sistema.

1.3 Casos de uso

ID	Caso de uso	Actor principal
CU01	Registrarse como paciente	Recepción / Paciente
CU02	Registrarse como doctor	Administrador

ID	Caso de uso	Actor principal
CU03	Consultar horarios disponibles	Recepción / Paciente
CU04	Agendar cita	Recepción
CU05	Cancelar cita	Recepción / Administrador
CU06	Reprogramar cita	Recepción / Administrador
CU07	Consultar calendario de citas	Doctor / Recepción
CU08	Consultar doctores disponibles	Paciente / Recepción
CU09	Administrar usuarios	Administrador
CU10	Modificar datos del paciente	Recepción / Administrador
CU11	Envío de notificación al paciente y al doctor	Sistema

1.4 Elección y evaluación de lenguajes

Python se mantiene como lenguaje principal porque permite combinar programación imperativa, orientada a objetos y funcional. Su sintaxis es legible, el tipado es dinámico y fuerte, la memoria se administra automáticamente y dispone de funciones de primera clase, módulos, clases, funciones y un ecosistema adecuado para aplicaciones web, bases de datos y pruebas.

JavaScript se utilizará en el frontend para eventos, validaciones y actualización dinámica de la interfaz. SQL se reserva para la persistencia relacional y HTML/CSS para estructura y presentación.

1.5 Distribución propuesta del sistema

Tecnología	Responsabilidad
Python	Lógica del servidor, reglas de negocio, autenticación y validación funcional.
JavaScript	Interacción del frontend, formularios, selección de horarios y consumo de la API.
SQL	Persistencia futura de usuarios, médicos, pacientes, horarios y citas.
HTML/CSS	Estructura y diseño visual de la aplicación web.

2. Selección de una funcionalidad

2.1 Funcionalidad seleccionada: búsqueda de horarios disponibles de un médico

La funcionalidad recibe la agenda de un médico, su horario laboral y la duración solicitada para una cita. El objetivo es calcular los espacios libres donde una nueva cita puede colocarse sin traslapes. Esta selección coincide con el pseudocódigo desarrollado en el avance previo y permite demostrar de forma clara el paradigma funcional.

Problema que resuelve: evita que recepción o el paciente tengan que revisar manualmente cada cita ocupada para localizar un bloque libre. La función transforma una lista de citas en una lista de intervalos disponibles.

Usuarios: la funcionalidad puede ser consultada por Recepción y Paciente en el caso de uso CU03. La agenda ficticia se utiliza como entrada del cálculo; no se requiere base de datos para esta etapa.

2.2 Relación con requerimientos y casos de uso

Elemento	Relación
R04	El resultado principal de la funcionalidad son los horarios disponibles que el sistema muestra al usuario.
R06	La disponibilidad calculada evita seleccionar un horario que se encuentre ocupado por otra cita.
CU03	Es el caso de uso principal: consultar horarios disponibles.
CU04	La información producida por esta funcionalidad puede utilizarse posteriormente para agendar una cita.

2.3 Entradas, transformaciones y salidas

Etapas	Contenido
Entradas	Lista de citas con inicio y duración; horario laboral; duración solicitada para una nueva cita.
Validación	Se comprueba que la duración sea positiva y que las citas existentes permanezcan dentro del horario laboral.
Transformación 1	Cada cita se convierte en un intervalo [inicio, fin].
Transformación 2	Los intervalos se ordenan por inicio y los que se traslapan se fusionan.
Transformación 3	Se calculan los huecos entre los intervalos ocupados.
Transformación 4	Cada hueco se divide en bloques consecutivos de la duración solicitada.
Salida	Lista de intervalos disponibles o un resultado de error.

2.4 Adecuación al paradigma funcional

La funcionalidad se presta al paradigma funcional porque puede expresarse como una cadena de transformaciones de datos. Las funciones reciben sus entradas explícitamente y devuelven nuevas estructuras sin depender de estado externo. Esto facilita razonar sobre cada etapa y reutilizar funciones pequeñas.

Ventajas frente a una solución imperativa: menor dependencia de variables mutables, separación clara entre transformaciones, posibilidad de usar composición y funciones de orden superior, y facilidad para probar cada función de manera aislada.

Limitaciones: el costo de crear nuevas colecciones puede aumentar el uso de memoria en escenarios muy grandes, y el estilo puede resultar menos familiar para quien sólo ha trabajado con ciclos y variables modificables. Para esta etapa y con datos ficticios pequeños, esas limitaciones no afectan el objetivo académico.

3. Diseño de la solución

3.1 Estructuras inmutables

Se utilizan tres estructuras principales. Cita representa una cita existente; Intervalo representa un periodo de tiempo; Resultado permite devolver Éxito o Error. En Python se implementan con dataclasses congeladas mediante `@dataclass(frozen=True)`, evitando la modificación posterior de sus atributos.

Tipo	Campos	Propósito
Cita	inicio, duracion	Representar una cita de la agenda.

Tipo	Campos	Propósito
Intervalo	inicio, fin	Representar tiempo ocupado o disponible.
Resultado	Exito(valor) / Error(mensaje)	Comunicar una salida válida o un error sin excepciones para reglas esperadas.

3.2 Funciones, entradas y salidas

Función	Entrada	Salida
calcular_fin	Cita	Entero
validar_cita	Cita, Intervalo	Resultado<Cita>
cita_a_intervalo	Cita	Intervalo
ordenar_por_inicio	Lista de intervalos	Nueva lista ordenada
fusionar	Lista de intervalos	Nueva lista fusionada
calcular_huecos	Lista de intervalos, horario	Lista de huecos
dividir_hueco	Intervalo, duración	Lista de intervalos
formatear_hora	Minutos	Texto HH:MM
buscar_disponibilidad	Citas, horario, duración	Resultado<Lista<Intervalo>>

3.3 Pseudocódigo

TIPO Cita = (inicio: Entero, duracion: Entero)
 TIPO Intervalo = (inicio: Entero, fin: Entero)
 TIPO Resultado = Exito(valor) | Error(mensaje)
 CONSTANTE HORARIO_LABORAL = Intervalo(480, 840)

FUNCION calcularFin(cita) -> Entero
 RETORNAR cita.inicio + cita.duracion
 FIN FUNCION

FUNCION validarCita(cita, horario) -> Resultado
 SI cita.duracion <= 0 ENTONCES
 RETORNAR Error("Duracion invalida")
 SI cita.inicio < horario.inicio O calcularFin(cita) > horario.fin ENTONCES
 RETORNAR Error("Cita fuera del horario laboral")
 RETORNAR Exito(cita)
 FIN FUNCION

FUNCION citaAIntervalo(cita) -> Intervalo
 RETORNAR Intervalo(cita.inicio, calcularFin(cita))
 FIN FUNCION

FUNCION ordenarPorInicio(intervalos) -> Lista
 RETORNAR copiaOrdenada(intervalos, clave = lambda i -> i.inicio)
 FIN FUNCION

FUNCION fusionar(intervalos) -> Lista

```

FUNCION acumular(acc, actual) -> Lista
  SI acc esta vacia ENTONCES RETORNAR [actual]
  ultimo <- ultimo elemento de acc
  SI actual.inicio <= ultimo.fin ENTONCES
    combinado <- Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
    RETORNAR (acc sin ultimo) + [combinado]
  RETORNAR acc + [actual]
FIN FUNCION

RETORNAR reducir(intervalos, acumular, [])
FIN FUNCION

```

```

FUNCION calcularHuecos(ocupados, horario) -> Lista
  puntos <- [Intervalo(horario.inicio, horario.inicio)] + ocupados
    + [Intervalo(horario.fin, horario.fin)]
  pares <- zip(puntos sin ultimo, puntos sin primero)
  candidatos <- mapear(pares, lambda (a,b) -> Intervalo(a.fin,b.inicio))
  RETORNAR filtrar(candidatos, lambda h -> h.fin > h.inicio)
FIN FUNCION

```

```

FUNCION dividirHueco(hueco, duracion) -> Lista
  cantidad <- division_entera(hueco.fin-hueco.inicio, duracion)
  RETORNAR mapear(rango(0,cantidad),
    lambda k -> Intervalo(hueco.inicio+k*duracion,
      hueco.inicio+(k+1)*duracion))
FIN FUNCION

```

```

FUNCION buscarDisponibilidad(citas, horario, duracion) -> Resultado
  validar duracion
  validaciones <- mapear(citas, lambda c -> validarCita(c, horario))
  SI existe error EN validaciones -> RETORNAR primer error
  espacios <- componer(
    lambda cs -> mapear(cs, citaAIntervalo),
    ordenarPorInicio,
    fusionar,
    lambda ocs -> calcularHuecos(ocs, horario),
    lambda hs -> aplanar(mapear(hs, lambda h -> dividirHueco(h, duracion)))
  )(citas)
  RETORNAR Exito(espacios)
FIN FUNCION

```

3.4 Conceptos funcionales aplicados

- Funciones puras: el resultado depende de las entradas y no del estado externo.
- Inmutabilidad: las citas y los intervalos originales no se modifican; se producen nuevas colecciones.
- Funciones de orden superior: map, filter, reduce, existe y componer reciben funciones.

- Lambdas: se usan como claves, filtros y transformaciones pequeñas.
- Composición: buscar_disponibilidad encadena varias etapas de transformación.
- Separación cálculo/E-S: la lógica de negocio está separada de la impresión y de la interfaz web.

4. Implementación

4.1 Organización del proyecto

```
gestor_citas_project/
├─ app.py
├─ INICIAR.bat
├─ abrir_frontend.bat
├─ ejecutar_pruebas.bat
├─ EJECUTAR.txt
├─ README.md
├─ data/
│   └─ citas_ficticias.json
├─ frontend/
│   ├── index.html
│   ├── styles.css
│   └─ app.js
├─ src/
│   ├── __init__.py
│   ├── gestor_citas.py
│   └─ servidor.py
├─ tests/
│   ├── __init__.py
│   └─ test_gestor_citas.py
└─ docs/
```

4.2 Ejecución en Python y Visual Studio Code

1. Abrir la carpeta completa del proyecto en Visual Studio Code.
2. Abrir Terminal > Nueva terminal.
3. Ejecutar python app.py. En Windows también se puede ejecutar py app.py.
4. El programa inicia un servidor local y abre el navegador en una dirección 127.0.0.1, buscando automáticamente otro puerto si el 8000 ya está ocupado.
5. En VS Code también existe una configuración de ejecución llamada “Gestor de Citas - Web”.
6. Las pruebas se ejecutan con python -m unittest discover -s tests -v o mediante la configuración “Pruebas - unittest”.

4.3 Frontend desarrollado

Se incorporó una interfaz web sin dependencias externas. El frontend utiliza HTML, CSS y JavaScript puro y presenta un flujo simple para consultar disponibilidad. El usuario puede seleccionar médico, fecha y duración, ejecutar la búsqueda, observar las citas ocupadas y seleccionar un horario disponible.

La interfaz consume dos rutas del servidor: GET /api/doctores para obtener el catálogo ficticio y GET /api/disponibilidad?doctor=...&duracion=... para solicitar el cálculo funcional realizado por Python.

Como respaldo para pruebas de interfaz, index.html también dispone de un modo de demostración local. Si se abre directamente o con Live Server y no existe conexión con Python, se cargan los mismos datos ficticios y el frontend puede mostrar resultados sin depender del servidor.

4.4 Datos ficticios

ID	Médico	Especialidad	Citas iniciales
MED001	Dra. Ana Lopez	Medicina general	09:00-09:30; 10:00-11:00; 10:30-11:00
MED002	Dr. Carlos Martinez	Cardiología	08:00-08:30; 09:30-10:00; 12:00-13:00
MED003	Dra. Sofia Hernandez	Pediatría	Sin citas

5. Pruebas

Se definieron 15 casos de prueba y se ejecutaron antes de cerrar la implementación. Incluyen escenarios normales, límites, entrada vacía, valores inválidos, traslapes, orden diferente de entrada y comprobación de inmutabilidad.

ID	Escenario	Entrada	Resultado esperado	Resultado obtenido	Estado
P01	Calcular final de una cita	Cita 09:00, duración 30	570 minutos	570 minutos	Aprobada
P02	Duración negativa	-15 minutos	Error de duración inválida	Error de duración inválida	Aprobada
P03	Duración cero	0 minutos	Error de duración inválida	Error de duración inválida	Aprobada
P04	Duración mayor a jornada	361 minutos	Error por exceder jornada	Error por exceder jornada	Aprobada
P05	Agenda vacía	Sin citas, 60 min	6 bloques disponibles	6 bloques disponibles	Aprobada
P06	Cita al inicio	08:00-08:30, 30 min	Primer bloque disponible 08:30	Primer bloque 08:30	Aprobada
P07	Cita al final	13:30-14:00, 30 min	Último bloque ocup./resto disponible	Resto de bloques disponible	Aprobada
P08	Dos citas separadas	09:00-09:30 y 11:00-11:30	Bloques libres entre citas	Bloques esperados	Aprobada
P09	Traslape	09:00-10:00 y 09:30-10:00	Fusionar intervalo ocupado	Intervalo fusionado	Aprobada
P10	Cita fuera de horario	13:50-14:10	Error fuera de horario	Error fuera de horario	Aprobada
P11	Citas fuera de orden	11:00 y 09:00	Ordenar sin cambiar entrada	Orden correcto	Aprobada
P12	Hueco no múltiplo	Hueco de 75 min, cita 30	2 bloques completos	2 bloques completos	Aprobada
P13	División de hueco	08:00-09:15, 30 min	08:00-08:30 y 08:30-09:00	08:00-08:30 y 08:30-09:00	Aprobada
P14	Inmutabilidad de fusionar	Dos intervalos traslapados	Lista original sin cambios	Lista original sin cambios	Aprobada
P15	Inmutabilidad de disponibilidad	Agenda ficticia	Entrada original sin cambios	Entrada original sin cambios	Aprobada

5.1 Resultado global

Las 15 pruebas terminaron en estado OK. El conjunto cubre validación de reglas, cálculo de huecos, fusión de traslapes, división por duración y verificación de inmutabilidad. También se verificó la compilación de los módulos Python sin errores sintácticos.

6. Repositorio de GitHub

El proyecto está organizado para utilizar un repositorio público por equipo durante el semestre. La estructura incluye código fuente, pruebas, datos ficticios, frontend y documentación.

Para preparar el repositorio: crear un repositorio público, subir la carpeta completa, conservar la estructura de directorios y realizar contribuciones con las cuentas personales de los integrantes. No se incluyen contraseñas, datos personales reales ni conexión a una base de datos real en esta etapa.

7. Conclusiones

La funcionalidad de búsqueda de horarios disponibles quedó diseñada e implementada con un flujo coherente entre requerimientos, casos de uso, pseudocódigo y código Python. La solución utiliza inmutabilidad y composición funcional para transformar la agenda del médico en intervalos disponibles.

La incorporación del frontend permite visualizar el resultado de forma clara sin alterar la separación del cálculo. La interfaz puede consumir el backend Python o utilizar un modo de demostración con datos ficticios, mientras que la lógica principal continúa concentrada en el módulo funcional y es verificable mediante 15 pruebas automatizadas.

Anexo A. Fragmento principal de implementación

```
def calcular_fin(cita: Cita) -> int:
    return cita.inicio + cita.duracion
```

```
def validar_cita(cita: Cita, horario: Intervalo) -> Resultado:
    if cita.duracion <= 0:
        return Error("Duracion invalida")
    if cita.inicio < horario.inicio or calcular_fin(cita) > horario.fin:
        return Error("Cita fuera del horario laboral")
    return Exito(cita)
```

```
def cita_a_intervalo(cita: Cita) -> Intervalo:
    return Intervalo(cita.inicio, calcular_fin(cita))
```

```
def ordenar_por_inicio(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    return tuple(sorted(intervalos, key=lambda intervalo: intervalo.inicio))
```

```

def fusionar(intervalos: Sequence[Intervalo]) -> Tuple[Intervalo, ...]:
    def acumular(
        acumulados: Tuple[Intervalo, ...], actual: Intervalo
    ) -> Tuple[Intervalo, ...]:
        if not acumulados:
            return (actual,)
        ultimo = acumulados[-1]
        if actual.inicio <= ultimo.fin:
            combinado = Intervalo(ultimo.inicio, max(ultimo.fin, actual.fin))
            return acumulados[:-1] + (combinado,)
        return acumulados + (actual,)

    return reduce(acumular, intervalos, ())

def calcular_huecos(
    ocupados: Sequence[Intervalo], horario: Intervalo
) -> Tuple[Intervalo, ...]:
    puntos = (
        Intervalo(horario.inicio, horario.inicio),
        *ocupados,
        Intervalo(horario.fin, horario.fin),
    )
    pares = zip(puntos[:-1], puntos[1:])
    candidatos = tuple(
        map(lambda par: Intervalo(par[0].fin, par[1].inicio), pares)
    )
    return tuple(filter(lambda hueco: hueco.fin > hueco.inicio, candidatos))

def dividir_hueco(hueco: Intervalo, duracion: int) -> Tuple[Intervalo, ...]:
    cantidad = (hueco.fin - hueco.inicio) // duracion
    return tuple(
        map(
            lambda k: Intervalo(
                hueco.inicio + k * duracion,
                hueco.inicio + (k + 1) * duracion,
            ),
            range(cantidad),
        )
    )

```

```

def formatear_hora(minutos: int) -> str:
    horas, mins = divmod(minutos, 60)
    return f'{horas:02d}:{mins:02d}'

def componer(*funciones: Callable) -> Callable:
    def pipeline(valor):
        return reduce(lambda actual, funcion: funcion(actual), funciones, valor)

    return pipeline

def buscar_disponibilidad(
    citas: Sequence[Cita], horario: Intervalo, duracion: int
) -> Resultado:
    """Calcula bloques disponibles sin modificar las citas de entrada."""
    if duracion <= 0:
        return Error("Duracion invalida")
    if duracion > horario.fin - horario.inicio:
        return Error("La duracion excede el horario laboral")

    validaciones = tuple(map(lambda cita: validar_cita(cita, horario), citas))
    errores = tuple(filter(lambda r: isinstance(r, Error), validaciones))
    if errores:
        return errores[0]

    pipeline = componer(
        lambda cs: tuple(map(cita_a_intervalo, cs)),
        ordenar_por_inicio,
        fusionar,
        lambda ocupadas: calcular_huecos(ocupadas, horario),
        lambda huecos: tuple(
            chain.from_iterable(map(lambda hueco: dividir_hueco(hueco, duracion), huecos))
        ),
    )
    return Exito(pipeline(tuple(citas)))

def intervalo_a_dict(intervalo: Intervalo) -> dict:
    return asdict(intervalo)

def resultado_a_dict(resultado: Resultado) -> dict:

```

```
if isinstance(resultado, Error):
    return {"ok": False, "error": resultado.mensaje}
return {
    "ok": True,
    "intervalos": [intervalo_a_dict(i) for i in resultado.valor],
    "horario": intervalo_a_dict(HORARIO_LABORAL),
}
```